Невронен машинен превод чрез генеративен езиков модел

Български → Английски

Асен Попов

Търсене и извличане на информация.
Приложение на дълбоко машинно обучение
Зимен семестър 2025/2026

February 14, 2026

Contents

1 Въведение **4**
1.1 Корпус . . . 4
1.2 Оценяване . . . 4
1.3 Платформа и инструменти . . . 4

2 Фаза 1: Базова Transformer архитектура **5**
2.1 Архитектура . . . 5
2.1.1 Основни компоненти . . . 5
2.1.2 Математическо описание . . . 5
2.2 Хиперпараметри . . . 6
2.3 Обучение . . . 6
2.4 Резултати . . . 6
2.5 Анализ . . . 6

3 Фаза 2: BPE токенизация **7**
3.1 Мотивация . . . 7
3.2 BPE токенизатор . . . 7
3.2.1 BPE алгоритъм . . . 7
3.3 Архитектура . . . 8
3.4 Критична грешка: Несъответствие на специалните токени . . . 8
3.4.1 Проблем . . . 8
3.4.2 Последствия . . . 8
3.4.3 Решение . . . 8
3.5 Резултати . . . 9
3.6 Сравнение с Фаза 1 . . . 9
3.7 Анализ . . . 9

4 Фаза 3: Convolutional Encoder + LSTM Decoder **10**
4.1 Мотивация . . . 10
4.2 Архитектура . . . 10
4.2.1 Encoder: Dual-Stack CNN . . . 10
4.2.2 Decoder: 3-Layer LSTM . . . 10
4.2.3 Luong Dot-Product Attention . . . 10
4.3 Хиперпараметри . . . 11
4.4 Обучение . . . 11
4.5 Резултати: Катастрофален провал . . . 11
4.6 Диагностика: Колапс на attention механизма . . . 12
4.6.1 Анализ на attention теглата . . . 12
4.7 Причина за провала . . . 12
4.8 Критично подобрение: Преминаване към Word-Level токенизация . . . 12
4.8.1 Откриване на проблема . . . 12
4.8.2 Нови параметри . . . 13
4.8.3 Нови резултати . . . 13
4.8.4 Анализ на подобрението . . . 13
4.9 Сравнение на всички експерименти . . . 14

5 Изводи и заключения **15**
5.1 Основни резултати 15

6 Използвани източници **16**

A Структура на проекта **17**

B Примерни команди **17**
B.1 Фаза 1: Базов Transformer 17
B.2 Фаза 2: BPE Токенизация 18
B.3 Фаза 3: CNN-LSTM 18

1 Въведение

Този проект реализира невронен машинен превод (Neural Machine Translation, NMT) от български към английски език чрез генеративен езиков модел. Проектът е изпълнен в три последователни фази, всяка от които изследва различна архитектура и техника за подобряване на качеството на превода.

1.1 Корпус

За обучение и тестване е използван двуезичен българско-английски подравнен корпус с текстове от Европейския парламент:

- **Обучение:** 180,000 двойки изречения (`train.bg`, `train.en`)
- **Валидация:** 1,000 двойки изречения (`dev.bg`, `dev.en`)
- **Тестване:** 6,000 двойки изречения (`test.bg`, `test.en`)

1.2 Оценяване

Качеството на преводите се оценява с два основни метрики:

1. **Перплексия (Perplexity):** Мярка за колко добре моделът предсказва валидационния корпус. Перплексията е експонентата на средната cross-entropy loss:

$$\text{PPL} = 2^H \quad \text{или} \quad \text{PPL} = e^H \tag{1}$$

   където H е средната cross-entropy:

$$H = -\frac{1}{N}\sum_{i=1}^{N} \log P(w_i|w_{<i}) \tag{2}$$

   В PyTorch, тъй като се използва natural logarithm, перплексията се изчислява като $\text{PPL} = \exp(H)$. По-ниската перплексия означава по-добро предсказване на следващия токен.

2. **BLEU Score:** Мярка за съответствието между машинния превод и референтния човешки превод (0-100, по-високо е по-добре).

1.3 Платформа и инструменти

- **Фреймуърк:** PyTorch 2.x
- **Хардуер:** NVIDIA GeForce RTX 4060 Laptop GPU (8GB VRAM)
- **Език:** Python 3.x

2 Фаза 1: Базова Transformer архитектура

2.1 Архитектура

Първата фаза използва **decoder-only Transformer** архитектура (в стил GPT), която моделира превода като задача за генериране на последователност от вида:

```
<S> [български текст] <TRANS> [английски текст] </S>
```

2.1.1 Основни компоненти

Компонент	Описание
Embedding слой	Научени токен влагания с padding маска
Позиционно влагане	Positional encoding
Attention	Multi-Head Self-Attention с каузална маска
Feed-Forward	2-слойна FFN с ReLU активация
Нормализация	Layer Normalization (post-norm)
Изход	Линейна проекция към речника

Table 1: Компоненти на Transformer модела

2.1.2 Математическо описание

Multi-Head Self-Attention:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \tag{3}$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{4}$$

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W_O \tag{5}$$

Transformer блок:

$$Z_1 = \text{LayerNorm}(X + \text{Dropout}(\text{MultiHead}(X))) \tag{6}$$

$$Z_2 = \text{LayerNorm}(Z_1 + \text{Dropout}(\text{FFN}(Z_1))) \tag{7}$$

Каузална маска: Долнотриъгълна маска, която предотвратява достъпа до бъдещи токени:

$$\text{mask}_{ij} = \begin{cases} 1 & \text{ако } i \geq j \\ 0 & \text{иначе} \end{cases} \tag{8}$$

2.2 Хиперпараметри

Параметър	**Стойност**
Размер на речника	57,341
Размерност на модела (d_{model})	256
Брой attention глави	8
Размерност на глава (d_k, d_v)	32
Размерност на FFN (d_{ff})	1,024 ($4 \times d_{\text{model}}$)
Брой Transformer слоеве	4
Dropout	0.1
Максимална дължина на последователност	5,000
Optimizer	Adam
Learning rate	0.0001
Batch size	8
Gradient clipping	1.0
Епохи	10

Table 2: Хиперпараметри на модела (Фаза 1)

2.3 Обучение

Време за обучение: ∼2.25 часа (225,000 итерации)

Токенизация: Word-level с праг на честота (думи, срещащи се $\leq$ 2 пъти $\rightarrow$ `<UNK>`)

Итерация	**Епоха**	**Loss**
100	1	8.45
1,000	1	6.36
10,000	1	4.89
100,000	5	3.21
225,000	10	2.54

Table 3: Прогрес на обучението (Фаза 1)

2.4 Резултати

Метрика	**Стойност**
Най-добра перплексия (dev)	16.33
Финална перплексия (dev)	16.39
BLEU score (test)	**21.77**

Table 4: Резултати от оценяването (Фаза 1)

2.5 Анализ

Силни страни:

- Стабилно намаляване на loss-а от 8.45 до 2.54
- Перплексия 16.33 показва добро моделиране на езика
- BLEU 21.77 е солидна базова линия

Слаби страни:

- Голям речник (57K) с много редки думи → `<UNK>`
- Български думи се появяват в английския изход (езиково смесване)
- Multinomial sampling въвежда случайност в генерирането

3 Фаза 2: BPE токенизация

3.1 Мотивация

Основният проблем на Фаза 1 е справянето с редки и непознати думи (`<UNK>`). Byte-Pair Encoding (BPE) решава този проблем, като разбива думите на подумови единици, което позволява представянето на всяка дума чрез комбинация от по-чести токени.

3.2 BPE токенизатор

Използва се собствена имплементация на BPE алгоритъма [6]:

- **Източник:** `github.com/marta1994/efficient_bpe_explanation`
- **Структури:** MaxPriorityMap, Linked_Array за ефективно сливане на двойки
- **Минимална честота на токен:** 100
- **Максимум итерации:** 20,000
- **Размер на речника:** 20,250 подумови единици

3.2.1 BPE алгоритъм

1. Инициализация на речника с всички символи
2. Повтаряне:
   (a) Намиране на най-честата двойка съседни токени (t_i, t_{i+1})
   (b) Създаване на нов токен $t_{\text{new}} = t_i \circ t_{i+1}$
   (c) Заместване на всички срещания на (t_i, t_{i+1}) с t_{new}
3. Спиране след достигане на максимум итерации

3.3 Архитектура

Архитектурата остава същата като Фаза 1 (decoder-only Transformer), но с по-големи размерности:

Параметър	Стойност
Размер на речника	20,250
Размерност на модела (d_{model})	512
Брой attention глави	4
Брой Transformer слоеве	5
Dropout	0.3
Batch size	24
Learning rate	0.0001
Епохи	20

Table 5: Хиперпараметри на модела (Фаза 2)

3.4 Критична грешка: Несъответствие на специалните токени

3.4.1 Проблем

BPE токенизаторът присвоява специалните токени на грешни индекси:

Токен	Очаквано от модела	Присвоено от BPE
`<S>`	0	20,247
`</S>`	1	20,248
`<UNK>`	2	246
`<PAD>`	3	20,249
`<TRANS>`	4	247

Table 6: Несъответствие на индексите на токените

3.4.2 Последствия

- Моделът не може да научи границите на изреченията
- Генерирането никога не спира на `</S>`
- Изходите са безсмислени

3.4.3 Решение

Добавена е функция `prepareDataBPE()` в `utils.py`, която:

1. Преназначава специалните токени на индекси 0-4
2. Премества всички BPE токени да започват от индекс 5
3. Създава `_old_to_new` маппинг за преобразуване

3.5 Резултати

Епохи	Перплексия	BLEU
10	59.65	—
15	40.00	15.54
20	32.18	18.85

Table 7: Прогрес на обучението (Фаза 2)

3.6 Сравнение с Фаза 1

Метрика	Фаза 1 (Word-level)	Фаза 2 (BPE)
Размер на речника	57,341	20,250
Перплексия	16.33	32.18
BLEU	**21.77**	**18.85**
Епохи	10	20
OOV обработка	`<UNK>` токен	Символно ниво

Table 8: Сравнение Фаза 1 vs Фаза 2

3.7 Анализ

Основни наблюдения:

- BPE намалява речника с ∼65% (от 57K на 20K)
- По-висока перплексия (32 vs 16) показва по-трудно моделиране
- По-нисък BLEU (18.85 vs 21.77) показва по-слабо качество на превода
- BPE изисква повече епохи за обучение (20 vs 10)

4 Фаза 3: Convolutional Encoder + LSTM Decoder

4.1 Мотивация

Фази 1 и 2 използват decoder-only архитектура, а за експеримента при Фаза 3 е реализира специализирана encoder-decoder архитектура с:

- CNN encoder за обработка на входния (български) език
- LSTM decoder за генериране на изходния (английски) език
- Luong attention механизъм за свързване между encoder и decoder

4.2 Архитектура

Базирана на работата на Gehring et al. (2016) "A Convolutional Encoder Model for Neural Machine Translation" [1] за CNN encoder и Luong et al. (2015) "Effective Approaches to Attention-based NMT" [5] за attention механизма.

4.2.1 Encoder: Dual-Stack CNN

Енкодерът се състои от две паралелни 8-слойни конволюционни мрежи:

- **CNN-Z:** Генерира ключове (keys) за attention scoring
- **CNN-V:** Генерира стойности (values) за attention

Конволюционен блок:

$$h^{(l)} = \tanh\left(\mathrm{Conv}(h^{(l-1)}) + h^{(l-1)}\right) \tag{9}$$

където residual връзката е вътре в tanh активацията.

Входно представяне:

$$e = \mathrm{Dropout}(\mathrm{WordEmb}(x) + \mathrm{PosEmb}(\mathrm{pos})) \tag{10}$$

4.2.2 Decoder: 3-Layer LSTM

Декодерът използва 3-слойна LSTM с input-feeding:

$$\tilde{h}_t = \mathrm{LSTM}([\mathrm{emb}(y_{t-1}), c_{t-1}], h_{t-1}) \tag{11}$$

$$d_t = W_d \tilde{h}_t + g_t \tag{12}$$

където c_{t-1} е context векторът от предишната стъпка, а g_t е входната дума влагания.

4.2.3 Luong Dot-Product Attention

$$d_t = W_d h_{t-1} + g_t \tag{13}$$

$$\mathrm{score}_t = d_t \cdot Z^T \tag{14}$$

$$\alpha_t = \mathrm{softmax}(\mathrm{score}_t) \tag{15}$$

$$c_t = \sum_i \alpha_{t,i} V_i \tag{16}$$

Изход:

$$P(y_t | y_{<t}, x) = \mathrm{softmax}(W_o[h_t; c_t] + b_o) \tag{17}$$

4.3 Хиперпараметри

Параметър	**Стойност**
Размер на речника	7,021
Размерност на модела (d_{model})	512
LSTM hidden size	1,024
CNN слоеве (на стек)	8
LSTM слоеве	3
CNN kernel size	3
Dropout	0.1
Label smoothing	0.1
Максимална дължина	512
Брой параметри	**∼70M**
Optimizer	Adam
Learning rate	0.0001
Batch size	32
Gradient clipping	1.0
Епохи	15

Table 9: Хиперпараметри на модела (Фаза 3)

Забележка: BPE с `min_occurrence=500` намалява речника до 7,021 токена.

4.4 Обучение

Време за обучение: ∼6-8 часа (15 епохи)

Метрика	**Начало**	**След 15 епохи**
Training Loss	6.84	4.00
Dev Perplexity	583.03	57.19

Table 10: Прогрес на обучението (Фаза 3)

4.5 Резултати: Катастрофален провал

Метод на декодиране	**BLEU**	**Качество**
Greedy Decode	4.99	Катастрофално лошо
Beam Search (k=4)	4.72	По-лошо от greedy

Table 11: Резултати от оценяването (Фаза 3)

Наблюдавани проблеми:

- Екстремна повторяемост на фрази

- Халюцинирано съдържание (случайни имена на държави)
- Неясни изходи с общи фрази от Европейския парламент
- Няма семантично съответствие с българския вход

4.6 Диагностика: Колапс на attention механизма

За анализа е разработен специален скрипт `diagnose_attention.py`.

4.6.1 Анализ на attention теглата

Метрика	Изречение 1	Изречение 2
Attention Entropy	0.209 / 4.625 (4.51%)	0.132 / 3.611 (3.65%)
Средно тегло	0.009804	0.027027
Стандартно отклонение	0.096088	0.158557
Максимално тегло	1.000000	1.000000

Table 12: Статистика на attention теглата

Ентропия съотношение 4-5% показва ТЕЖЪК attention колапс (равномерното разпределение би дало ∼100%)

4.7 Причина за провала

Моделът се научи да игнорира енкодера и да функционира като чист езиков модел:

1. **Силен декодер:** 3-слойна LSTM с 1024 hidden units може да запомни целевото езиково разпределение
2. **Attention колапс:** Dot-product attention без скалиране се насища по време на обучението
3. **Динамика на обучението:** Моделът намери по-нисък loss, като запомни EU Parliament patterns, вместо да научи превод
4. **Липса на attention supervision:** Няма механизъм за налагане на attention coverage или alignment

4.8 Критично подобрение: Преминаване към Word-Level токенизация

4.8.1 Откриване на проблема

След неуспеха с BPE токенизация (BLEU 4.72), беше направен експеримент с връщане към word-level токенизация при същата CNN-LSTM архитектура. Резултатът беше драматично подобрение.

4.8.2 Нови параметри

Параметър	**Стойност**
Размер на речника	57,341
Размерност на модела (d_{model})	256
LSTM hidden size	512
CNN слоеве (на стек)	4
LSTM слоеве	2
CNN kernel size	3
Dropout	0.1
Максимална дължина	512
Брой параметри	**∼50.6M**
Optimizer	Adam
Learning rate	0.0001
Batch size	8
Gradient clipping	1.0
Епохи	15

Table 13: Хиперпараметри на модела с word-level токенизация (Фаза 3 - подобрена)

4.8.3 Нови резултати

Метрика	**BPE (провал)**	**Word-level (нова)**
Размер на речника	7,021	57,341
CNN слоеве	8	4
LSTM слоеве	3	2
d_model	512	256
hidden_size	1,024	512
Параметри	∼70M	∼50.6M
Batch size	32	8
Перплексия	57.19	**11.36**
BLEU (beam search, k=5)	**4.72**	**29.79**
Качество	Катастрофално	**Отлично**

Table 14: Сравнение BPE vs Word-level при CNN-LSTM архитектура

Подобрение: BLEU скок от 4.72 на 29.79 !

4.8.4 Анализ на подобрението

Защо BPE работеше добре във Фази 1-2, но не и във Фаза 3?

- **Фази 1-2 (decoder-only):** Attention беше self-attention върху смесена последователност от източник и цел. По-дългите BPE последователности не бяха критичен проблем.

- **Фаза 3 (encoder-decoder):** Cross-attention между два отделни корпуса. BPE създаваше различни подумови фрагменти за двата езика, правейки подравняването много по-трудно. Допълнително, по-големият модел (70M параметри) в комбинация с по-малкия BPE речник (7K) създаде дисбаланс.

4.9 Сравнение на всички експерименти

Метрика	**Ф1**	**Ф2**	**Ф3-BPE**	**Ф3-Word**
Архитектура	Dec-only	Dec-only	Enc-Dec	Enc-Dec
Токенизация	Word	BPE	BPE	Word
Attention тип	Self	Self	Cross	Cross
Речник	57,341	20,250	7,021	57,341
Параметри	∼15M	∼40M	∼70M	∼50M
Перплексия	16.33	32.18	57.19	**11.36**
BLEU	**21.77**	**18.85**	**4.72**	**29.79**
Епохи	10	20	15	15
Време (ч)	∼3-4	∼5-6	∼6-8	∼6-8
Качество	Добро	Средно	Провал	**Най-добро**

Table 15: Пълно сравнение на всички направени експерименти

5 Изводи и заключения

5.1 Основни резултати

1. **Фаза 1 (decoder-only Transformer):** Добра производителност с BLEU 21.77 - солидна базова линия

2. **Фаза 2 (BPE токенизация):** По-компактен речник, но по-ниско качество (BLEU 18.85)

3. **Фаза 3-BPE (CNN-LSTM с attention):** Катастрофален провал поради attention collapse и неподходяща токенизация (BLEU 4.72)

4. **Фаза 3-Word (CNN-LSTM с word-level): Най-добър резултат - BLEU 29.79** - encoder-decoder архитектурата работи отлично с правилната токенизация и параметри

6 Използвани източници

References

[1] J. Gehring, M. Auli, D. Grangier, and Y. N. Dauphin, "A convolutional encoder model for neural machine translation," *arXiv preprint arXiv:1611.02344*, 2016. https://arxiv.org/abs/1611.02344

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017. https://arxiv.org/abs/1706.03762

[3] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," *ICLR 2015*. https://arxiv.org/abs/1409.0473

[4] J. Gehring, M. Auli, D. Grangier, D. Yarats, and Y. N. Dauphin, "Convolutional sequence to sequence learning," *ICML 2017*. https://arxiv.org/abs/1705.03122

[5] M.-T. Luong, H. Pham, and C. D. Manning, "Effective approaches to attention-based neural machine translation," *EMNLP 2015*. https://arxiv.org/abs/1508.04025

[6] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," *ACL 2016*. https://arxiv.org/abs/1508.07909

[7] D. Britz, A. Goldie, M.-T. Luong, and Q. Le, "Massive exploration of neural machine translation architectures," *EMNLP 2017*. https://www.aclweb.org/anthology/D17-1151

A Структура на проекта

```
finalProject/
          en_bg_data/
                train.{bg,en}  # Training corpus (180k pairs)
                dev.{bg,en}    # Validation corpus (1k pairs)
                test.{bg,en}   # Test corpus (6k pairs)

          models/
                model.py              # Transformer model (Phase 1&2)
                convenc_lstmdec.py    # CNN-LSTM model (Phase 3)
                seq2seq_data.py       # Seq2seq data loaders

          shared/
                run.py           # Main training/testing script
                parameters.py    # Hyperparameters
                utils.py         # Data preparation functions
                tokenizer.py     # BPE tokenizer

          phase1_transformer/
                NMTmodel         # Trained model (Phase 1)
                wordsData        # Word-level vocabulary

          phase2_transformer_bpe/
                NMTmodel_bpe     # Trained model (Phase 2)
                bpeModel         # BPE model
                wordsData_bpe    # BPE vocabulary

          phase3_cnn_lstm/
                NMTmodel_convenc_lstmdec  # Trained model
                diagnose_attention.py     # Attention diagnostics
                corpusData_seq2seq_bpe    # Seq2seq data

          results/
              phase1_result.en
              phase2_result.en
              phase3_result_greedy.en
              phase3_result_beam.en
```

Listing 1: Файлова структура на проекта

B Примерни команди

B.1 Фаза 1: Базов Transformer

```
# Prepare data
python run.py prepare

# Train model
```

```
python run.py train

# Translate test set
python run.py translate en_bg_data/test.bg results/phase1_result.en

# Evaluate BLEU
python run.py bleu en_bg_data/test.en results/phase1_result.en
```

B.2 Фаза 2: BPE Токенизация

```
# Prepare BPE data
python run.py prepare_bpe

# Train model
python run.py train

# Translate test set
python run.py translate en_bg_data/test.bg results/phase2_result.en

# Evaluate BLEU
python run.py bleu en_bg_data/test.en results/phase2_result.en
```

B.3 Фаза 3: CNN-LSTM

```
# Prepare seq2seq data with BPE
python run.py prepare_seq2seq_bpe 500

# Train model
python run.py train_phase3

# Translate (greedy)
python run.py translate_phase3 en_bg_data/test.bg \
    results/phase3_result_greedy.en

# Translate (beam search)
python run.py translate_phase3 en_bg_data/test.bg \
    results/phase3_result_beam.en beam

# Diagnose attention
python phase3_cnn_lstm/diagnose_attention.py
```